

Three voice samples

LC-STRATEGY-V4.0.0 · Selected strategy. Complete proposed copy; exact assets await approval.
Sunil examples are illustrative until product-story evidence is cleared.

Roadmap and launch gates

Audience: Experienced engineer or technical lead · Practical question

Caption

The roadmap has a launch date. Does it have a reason to believe the system will be ready?

Imagine reviewing an agent that prepares a daily plan. The demo works. The team can explain the architecture. The calendar says it is time for a pilot.

Then you ask what happens when the inputs are incomplete. Or when the tool returns something the system cannot use. The answers are still being worked out.

That is the conversation I would want before agreeing to the date.

A date helps people coordinate. A release gate explains what must be true: which behaviour has been tested, what evidence is available, who owns the decision and what happens if the condition is missed.

I would put both on the roadmap. I would also name what we are deliberately leaving out. Otherwise, an extra week can become an extra feature instead of time to resolve the original uncertainty.

For this illustrative system, the next useful milestone might be a bounded trial with a review owner, rather than a wider release.

Look at one milestone on your own roadmap. Could someone outside the team explain what evidence would close it?

Model selection

Audience: Experienced engineer or technical lead · Practical question

Caption

“Which model should we use?” can hide several different decisions.

Imagine a system with a planner, a checker and a writer. The planner chooses a path. The checker examines constraints. The writer explains the result.

Giving them all the same model is convenient. It may also mean paying for capability a step does not need—or asking a model to do a check that code could perform more reliably.

I would compare the jobs before comparing the models. Then I would test candidate approaches on the work each role actually does.

A second model can offer another view, but a different vendor or model family does not by itself prove independent judgment.

For one role in your system, write down what would make you change your current choice.

Panel 1 — Choose for the job

A planner, checker and writer can have different needs. Start by naming the decision each one owns.

Panel 2 — Define acceptable behaviour

What must this role get right? Which failure would make the output unusable?

Panel 3 — Compare on your cases

Use representative inputs and the same acceptance criteria. Include difficult and incomplete cases.

Panel 4 — Count the surrounding cost

Include retries, review time and failed work alongside model charges. A cheaper call can cost more overall.

Panel 5 — Test the checker too

A second model can share mistakes. Compare its judgments with known cases and qualified human review.

Panel 6 — Make the choice revisitable

Record the reason, the limitation and the evidence that would trigger a new comparison.
Illustrative model-selection review.

Whether an agent is needed

Audience: Experienced engineer or technical lead · Practical question

Caption

Before choosing an agent framework, I would ask what actually needs an agent.

This illustrative planning example starts with a task that sounds like one job. Looking closely reveals several different jobs—and some are better handled by ordinary code.

Try the same separation on your next proposal: where is judgment needed, and where would a rule be enough?

Spoken script

Imagine your team is building a service that prepares a daily plan. Someone suggests an agent, and the conversation immediately turns to models and frameworks.

I would pause one step earlier. What decisions are we asking the system to make?

Checking whether a required field is present does not need a model. Neither does enforcing a fixed limit. Those are jobs we can describe and test directly.

Choosing between several reasonable options, with incomplete context, is a different problem. That is where I would examine whether a model adds useful judgment—and whether an agent needs tools to gather information or act.

I would also ask what happens when the answer is wrong. Can someone review it? Can we recover? How will we tell whether the result was useful?

In this illustrative example, the design might end up with a small agentic part surrounded by quite ordinary software. That is a reasonable outcome.

The question is whether the approach earns its complexity for this task.

Take one proposed agent in your team. Separate the fixed rules, the uncertain decisions and the permitted actions. Which part still needs an agent after you have done that?